



LEADERSHIP
COMPUTING
FACILITY

March 30, 2026 | WCISCC 2026

Quantum Transfer Learning Challenge

Dr. Michael Sandoval

National Center for Computational Sciences



U.S. DEPARTMENT
of **ENERGY**

ORNL IS MANAGED BY UT-BATTELLE LLC
FOR THE US DEPARTMENT OF ENERGY

FRONTIER



Agenda

Who are we?

**Your OLCF
“mystery”
mentors**

**The Quantum
Computing
User Program
& the QSC**

**What is
Quantum
Transfer
Learning?**

**How to use
Odo & IQM**

**The Challenge
Problem &
Demo**

OLCF “Mystery” Mentors



Michael
Sandoval



Jordan
Winetrout



Murali
Gopalakrishnan Meena



Suzanne
Parete-Koon

QCUP & the QSC

Quantum Computing at ORNL

Quantum Computing User Program

Enable Research

Provide a broad spectrum of user access to the best available quantum computing systems



Evaluate Technology

Monitor the breadth and performance of early quantum computing applications



Engage Community

Support growth of the quantum ecosystem by engaging with users, developers, vendors, and providers

QCUP Operations Model

User Assistance (UA) Group

- Provides technical support, training, documentation, and tools to users.

Software Services Development (SSD) Group

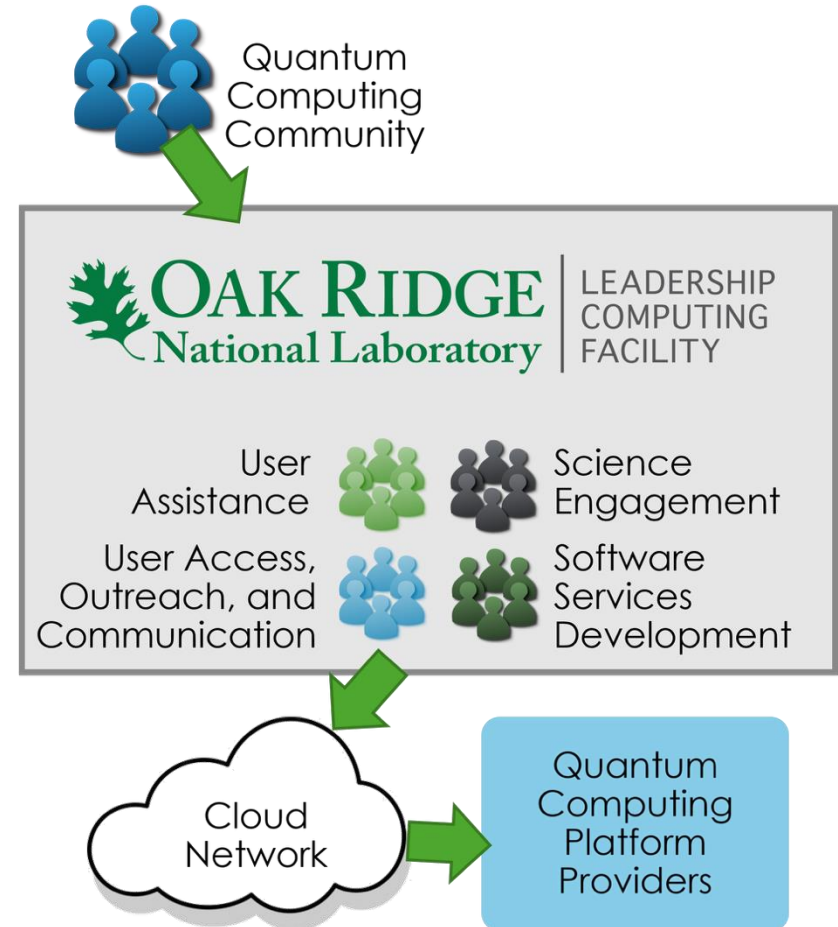
- Designs and builds custom software to support users and projects.
- Integrates vendor APIs into the center's existing software, automating various processes.

User Access, Outreach, and Communication Group

- Adds users and projects to vendor management systems.

Science Engagement (SE) Section

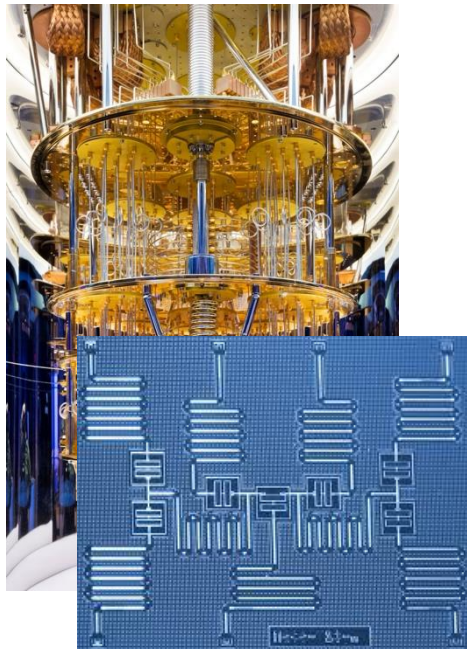
- Scientists that review proposals to the program and run workloads on QCUP machines for testing and research purposes.



Current QCUP Resources

IBM Quantum

7 superconducting transmon systems, up to 156 qubits.



IQM

3 superconducting transmon systems, up to 54 qubits.



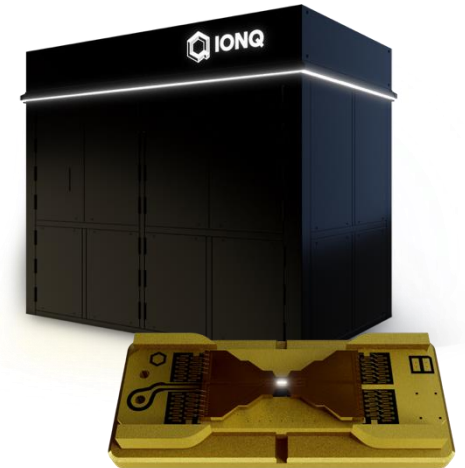
Quantinuum

3 trapped-ion systems, up to 98 qubits.



IonQ

3 trapped-ion systems, up to 36 qubits.



The Quantum Science Center (QSC)

- The Quantum Science Center is one of five DOE National Quantum Information Science Research Centers (NQISRCs) authorized by the National Quantum Initiative Act.
- Currently, the QSC is focused on accelerating quantum simulations of materials by leveraging high-performance computing alongside experimental validation of quantum materials.



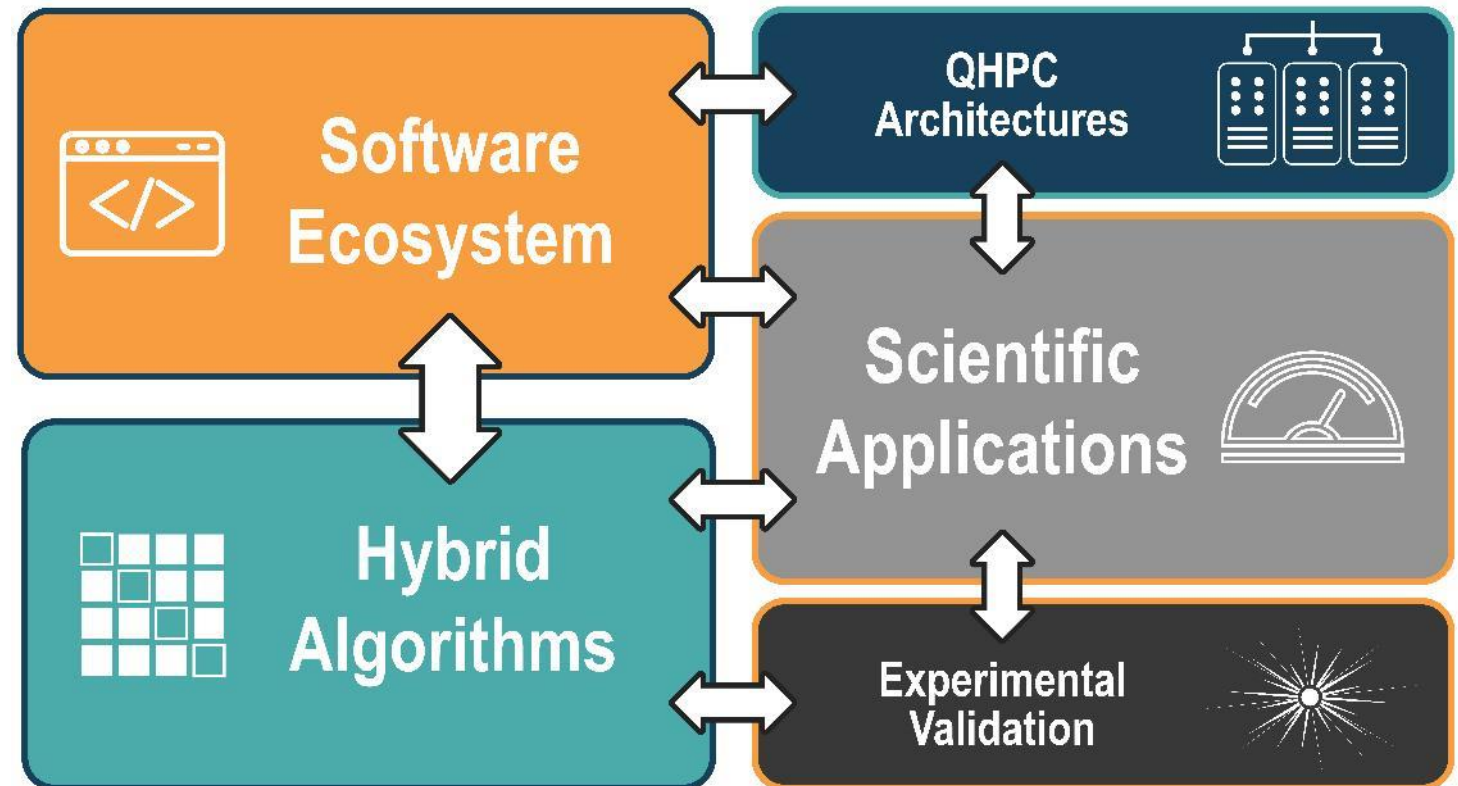
The QSC is creating a scientific ecosystem for quantum-accelerated high-performance computing (QHPC)

The **key deliverables** for the center are:

- openQSE, an open QHPC software ecosystem
- Hybrid algorithm resource estimations
- QHPC architecture specifications
- Scientific application libraries for materials simulations
- Database of materials measurements for validation
- QHPC training curriculum

The center consists of 3 national labs, 8 universities & 9 industry partners

- | | | |
|---|------------------|----------------------|
| • Oak Ridge National Laboratory | • UC Berkeley | • IonQ |
| • Los Alamos National Laboratory | • UC Davis | • IQM |
| • Pacific Northwest National Laboratory | • UC San Diego | • Quantinuum |
| • Caltech | • UT Knoxville | • QuEra |
| • Cornell University | • Atom Computing | • Riverlane |
| • North Carolina State University | • IBM | • Unitary Foundation |
| • Purdue University | • Inflektion | |



What is Quantum Transfer Learning

Quantum Background & Overview of Code

Quantum Primer

- Please see the complete Quantum Primer here for the full quantum background:
 - Slides: <https://github.com/olcf/hands-on-with-odo/tree/master/presentations>
 - Recording: <https://vimeo.com/1157625397>
 - Beginner friendly and starts from 0 quantum knowledge
- Only providing a very brief background in these WCISCC slides

What is Quantum Computing?

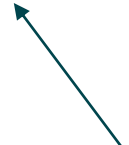
- In Quantum Mechanics, a wave function (“state”) describes knowledge of a system
- Quantum Computing (**QC**) involves manipulating this wave function to do computation while taking advantage of quantum “effects”
 - **Entanglement**: correlations between different parts of quantum system
 - “spooky action at a distance”
 - **Superposition**: representing multiple possible system configurations at the same time

Schrödinger Equation

$$i\hbar \frac{\partial}{\partial t} \Psi(t) = H(t) \Psi(t)$$



Wave function



Energy / “Hamiltonian”

What is a Qubit?

- A qubit is the quantum equivalent of a classical bit!
- While a classical bit can only be 0 or 1, a qubit can represent 0 and 1 at the same time!
 - With N qubits you can represent 2^N configurations! (e.g., for $N=50 \rightarrow 1,125,899,906,842,624$)
 - 1 qubit: 0, 1
 - 2 qubits: 0 0, 0 1, 1 0, or 1 1
 - Helps solve problems that scale exponentially
 - Nifty trick, but introduces “noise” (will talk about this more later)
- Physical representation depends on the technology
- Classical computers can only simulate so many qubits before running out of memory... (see image in 2 slides)

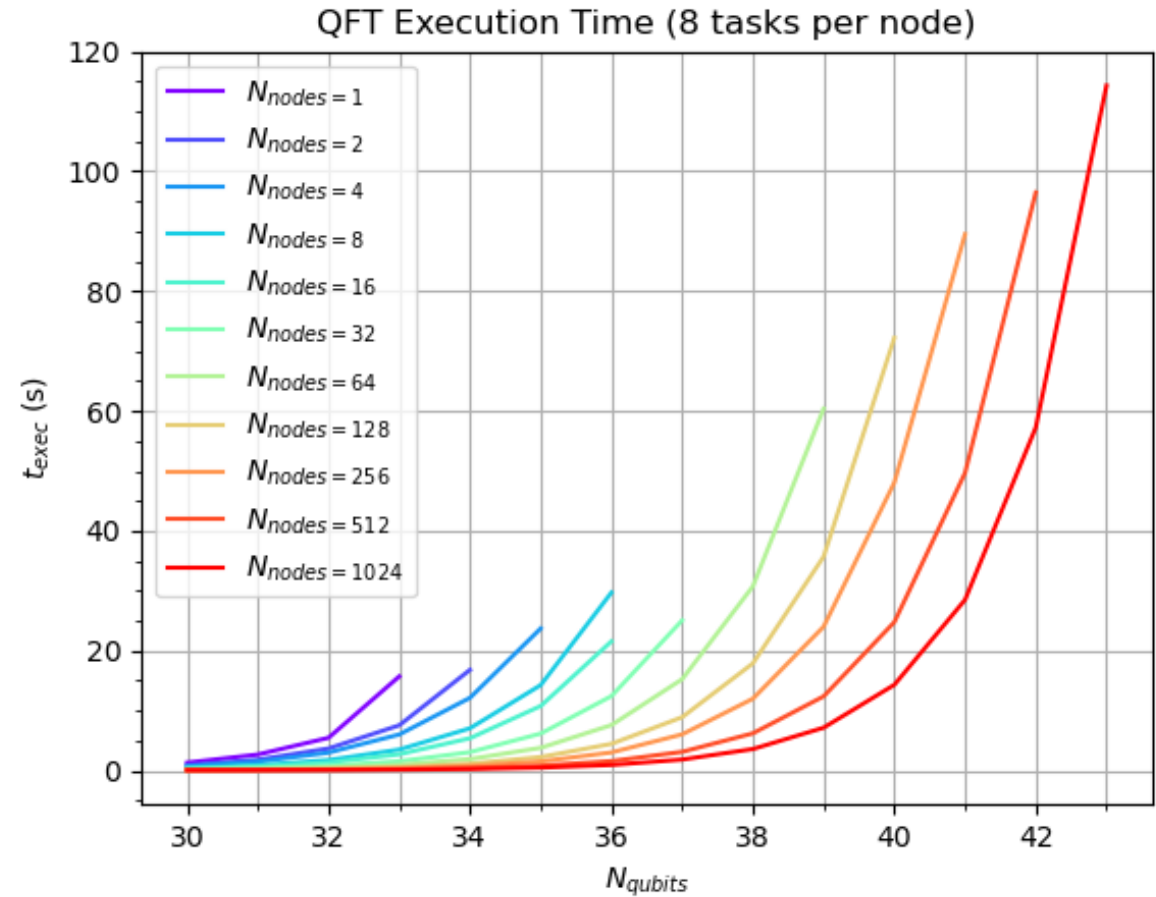
Options to run Quantum Computations

Backend	Mechanism	Functionality
Simulator	Classical	Classical program modeling a quantum system in an ideal scenario
Emulator	Classical	Classical program modeling actual behavior of a quantum system
Real	Quantum	Physical hardware performing real quantum computations

For the challenge, we'll be using Odo (classical) and IQM (real quantum hardware)

Example Scaling

- Each curve runs configurations until running out of memory
- Doubling the node count roughly lets you simulate 1 more qubit
- You can only get so far with classical computers before you quickly run out of nodes!
- And this is just for a “toy” problem!



“Regular” Transfer Learning

- In a nutshell, transfer learning is:
 - A machine learning technique where you repurpose a pre-trained model
 - Freeze early layers that identifies general features
 - Let the final layers train on a smaller, related dataset to fine-tune on specific features
 - You’re “transferring” the learning from one task and using it for another task

Quantum Motivation

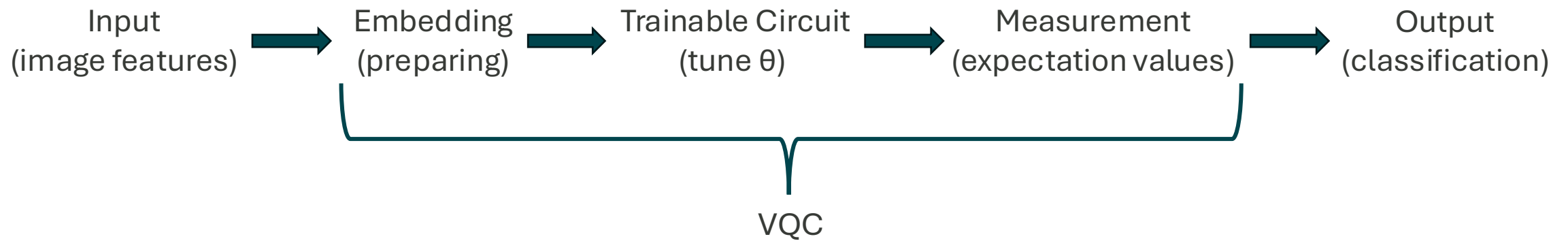
- Classical layers use standard math, while a quantum circuit mixes information using superposition, interference, and entanglement
 - This can extract more complex features not capturable by a classical network
- For some natural phenomena, it can be harder for a classical computer to simulate or approximate at scale -- nature is inherently “quantum”
- Using a quantum circuit may use less parameters to train than a classical network
 - For transfer learning specifically, can live in the realm of smaller qubits because smaller targeted training
- **Realistic Note**: in this era of quantum computing (QC), a purely classical model can match or beat these “hybrid” QC+Classical demos
 - **The key takeaway** is to explore where quantum could add value, and to pathfind QC/HPC use-cases as hardware improves
- We (you) will be investigating using a variational quantum circuit (VQC) in transfer learning and how viable it is

Variational Quantum Circuits (VQCs)

- A VQC is an adjustable quantum program with some knobs you can tune or train
 - Knobs: real numbers (parameters)
 - Tuning: changing the parameters so the circuit's output helps solve a task (e.g., image classification)
- Recall that quantum programs are composed of quantum “gates”
- So, what are the knobs/parameters?
 - Rotation gates require an angle (θ) as input
 - Adjusting θ can help minimize loss (classification error)
 - Like weights in a neural network, θ values can be trained via machine learning
- A VQC will be a layer we implement within an existing ML model

VQC Process

- The big picture



Embedding (Initializing Data in Qubits)

- Quantum circuits act on quantum states, not directly on classical numbers. So, we must encode the input features into a quantum state.

- Preparing your initial state is common among QC problems

- Approach in Mari et al. 2019

- Apply a Hadamard gate (create equal superposition/probabilities across all qubits)
 - Apply rotation gates where the angle (θ) is initially set by the input features of the incoming image

```
def H_layer(nqubits):  
    """Layer of single-qubit Hadamard gates.  
    """  
    for idx in range(nqubits):  
        qml.Hadamard(wires=idx)
```



```
def RY_layer(w):  
    """Layer of parametrized qubit rotations around the y axis.  
    """  
    for idx, element in enumerate(w):  
        qml.RY(element, wires=idx)
```



- What this does

- Creates a balanced starting point for all qubits
 - “Writes” or “encodes” the image data into a quantum state with θ

Trainable Circuit (The “Learning” Portion)

- This is the “neural network” part of the circuit, but with quantum
- After the initial embedding layers, the circuit applies more layers
 - Entangling gates (CNOT gates) that link qubits together
 - More single-qubit rotations (R_y gates) -- these are the trainable knobs
 - These blocks of layers may be repeated multiple times (quantum depth)
- What this does
 - Rotations make things learnable
 - Entanglement lets features influence each other
 - Depth leads to a more expressive model (sometimes)
 - Like a regular neural net, “more layers = better” in general, but can lead to overfitting if not careful

```
def entangling_layer(nqubits):  
    """Layer of CNOTs followed by another layer of CNOTs  
    """  
    # In other words it should apply so  
    # CNOT CNOT CNOT CNOT... CNOT  
    # CNOT CNOT CNOT... CNOT  
    for i in range(0, nqubits - 1, 2):  
        qml.CNOT(wires=[i, i + 1])  
    for i in range(1, nqubits - 1, 2):  
        qml.CNOT(wires=[i, i + 1])
```

```
# Sequence of trainable variational layers  
for k in range(q_depth):  
    entangling_layer(n_qubits)  
    RY_layer(q_weights[k])
```

Measurement (Extracting Data from Qubits)

- Must measure the qubits to extract data from the quantum state
 - Measurements result in probabilities
 - For our code, we take these probabilities and compute “expectation values”
 - Expectation values are weighted averages of measuring a specific observable
 - In our code, computing the expectation value for a qubit results in a value between $[-1, 1]$
 - If we measure 4 qubits, you can get 4 output numbers like: $[-.2, .8, .1, -.3]$
- These outputs become input features for the final classical layer (next slide) that converts things to scores/logits
- What this does
 - Retrieves data to be transformed later for classification
 - You can repeat this over multiple shots to refine the expectation values

```
# Expectation values in the Z basis
exp_vals = [qml.expval(qml.PauliZ(position)) for position in range(n_qubits)]
return tuple(exp_vals)
```



Dressed Quantum Circuit

- The VQC is combined with a classical pre-processing layer and classical post-processing layer
- First classical layer reshapes input features to match number of qubits
- Final classical layer maps the quantum outputs to the number of classes
- What this does

- Compresses initial data to the size of our quantum system
- Converts quantum data back to the size of our classes

```
class DressedQuantumNet(nn.Module):  
    """  
    Torch module implementing the *dressed* quantum net.  
    """  
  
    def __init__(self, device):  
        """  
        Definition of the *dressed* layout.  
        """  
  
        super().__init__()  
        self.pre_net = nn.Linear(512, n_qubits)  
        self.q_params = nn.Parameter(q_delta * torch.randn(q_depth * n_qubits))  
        self.post_net = nn.Linear(n_qubits, 2)
```

$$\tilde{Q} = L_{n_q \rightarrow n_{\text{out}}} \circ Q \circ L_{n_{\text{in}} \rightarrow n_q}$$

The Workflow of Our Problem Specifically

- Freeze ResNet18 model (previously trained on the ImageNet dataset)
- Replace final layer with Dressed Quantum Circuit
- Classify subset of ImageNet (ants/bees)
 - 245 training images, 153 validation images
 - Each image has 512 features associated when we take over from the pre-trained model
- Using 4 qubits, so size transformations looks like: $512 \rightarrow 4 \rightarrow 2$
- MPI can further split up the work but will affect accuracies
- Uses PyTorch, mpi4py, and PennyLane
 - Run things using either simulator or real quantum device

PennyLane Devices

- PennyLane is a cross-platform Python library for quantum computing, quantum machine learning, and quantum chemistry
 - <https://pennylane.ai/>
- Allows use of different quantum simulators
 - Solves expectation values analytically by default (ideal scenario), but can still generate sample distributions by passing a number of shots (introduces noise)
- Devices we will be exploring
 - lightning.kokkos (let's us run on AMD GPUs through the Kokkos library)
 - qiskit.remote (let's us run on real quantum backends)
- For a more complete demonstration of PennyLane running on Frontier, please see recording here: https://github.com/olcf/quantum-training-series/tree/main/pennylane_frontier_2025

Using Odo & IQM

Odo & IQM Resonance Overview

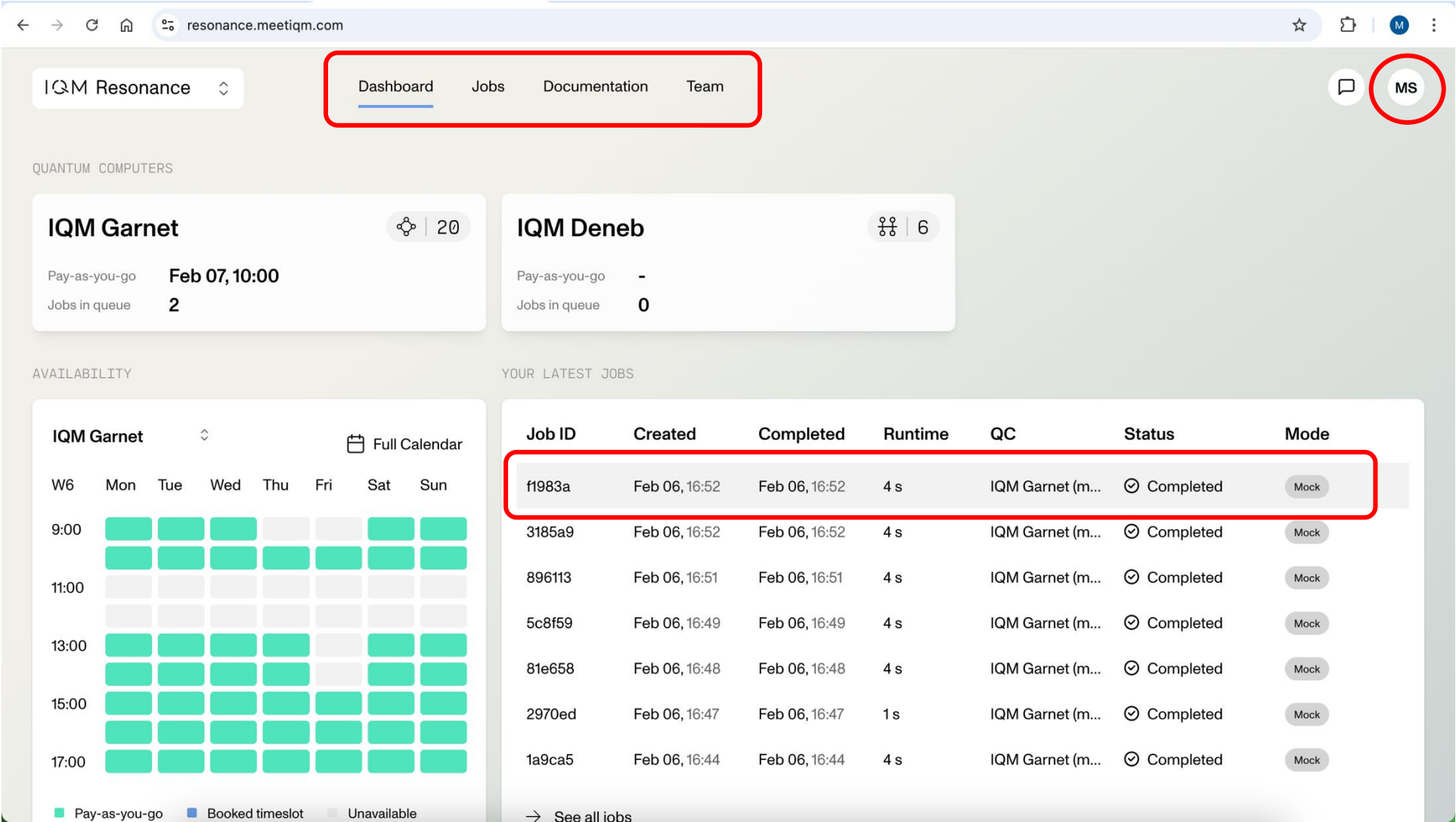
The Odo Cluster

- Odo is a 30 node training system based on the Frontier supercomputer
 - HPE Cray EX Supercomputer architecture
 - 3rd Gen AMD EPYC 64-core CPU per node
 - 4 AMD Instinct MI250X GPUs per node (2 GCDs per GPU – so effectively 8 GPUs per node)
- Using the same TRN044 project as you did last time
- As you've used this cluster in the last challenge, won't go over much on Odo here
- We will be using this for the classical part of our code (purely classical portions / quantum simulator portions)

IQM Resonance

- IQM provides access to quantum computing resources through their cloud platform, IQM Resonance.
 - The 54 qubit Emerald, the 20 qubit **Garnet** , and the 16 qubit Sirius systems.
 - IQM's quantum processors are made up of superconducting transmon qubits, utilizing multiple topologies.
 - User dashboard: <https://resonance.meetiqm.com/>
- OLCF Documentation: https://docs.olcf.ornl.gov/quantum/quantum_systems/iqm.html
- We'll be submitting jobs to Garnet via Python from Odo
 - "Timeslot" mode
 - Reserved for April 2nd and April 3rd at 3-4 PM (Eastern)
 - Big portion of your overall score, don't miss these timeslots! (and a chance to run on real quantum devices)
- Should've received an invite from me to create your account (**not the same account as Odo!!**)

The IQM Resonance Dashboard - I



The IQM Resonance Dashboard - II

← → ↺ 🏠 🔍 resonance.meetiqm.com ☆ 📁 M ⋮

IQM Resonance

Dashboard Jobs Documentation Team

IQM Garnet

Feb 01, 2026

IQM Details

Feb 01, 2026

IQM Garnet

Full Calendar

Job ID	Created	Completed	Runtime	QC
10000	Feb 01, 2026	Feb 01, 2026	0h	IQM Garnet
10001	Feb 01, 2026	Feb 01, 2026	0h	IQM Garnet
10002	Feb 01, 2026	Feb 01, 2026	0h	IQM Garnet
10003	Feb 01, 2026	Feb 01, 2026	0h	IQM Garnet
10004	Feb 01, 2026	Feb 01, 2026	0h	IQM Garnet
10005	Feb 01, 2026	Feb 01, 2026	0h	IQM Garnet
10006	Feb 01, 2026	Feb 01, 2026	0h	IQM Garnet
10007	Feb 01, 2026	Feb 01, 2026	0h	IQM Garnet
10008	Feb 01, 2026	Feb 01, 2026	0h	IQM Garnet
10009	Feb 01, 2026	Feb 01, 2026	0h	IQM Garnet
10010	Feb 01, 2026	Feb 01, 2026	0h	IQM Garnet

Your account

MS

Michael Sandoval

sandovalma@ornl.gov

T

Team

TRN037

API Token

expires Feb 6, 2026

🔄

Email preferences

Required service notifications

IQM newsletter

Log out

The “Mystery” Challenge

Demo

Perform Study of Quantum Transfer Learning on Simulators, Emulators, and Real Devices

- Demo Time
- Expanded version of a “Hands on With Odo” challenge
 - https://github.com/olcf/hands-on-with-odo/tree/master/challenges/Python_QML_Basics
 - If you cloned this prior to March 23rd, you need to clone again or run a `git pull` to update things
- Four Tasks
 1. Parameter Sweep
 2. Backend Evaluation
 3. Data Expansion
 4. Quantum Takeaways
- Detailed instructions included in PDF doc – see GitHub repo:

https://github.com/olcf/hands-on-with-odo/tree/master/challenges/Python_QML_Basics/wcisc2026

References, Funding, and Acknowledgments

Andrea Mari, Thomas R. Bromley, Josh Izaac, Maria Schuld, and Nathan Killoran. Transfer learning in hybrid classical-quantum neural networks. <https://arxiv.org/abs/1912.08278> arXiv:1912.08278 (2019).

Andrea Mari, "Quantum transfer learning tutorial".
https://pennylane.ai/qml/demos/tutorial_quantum_transfer_learning/

Sasank Chilamkurthy, "PyTorch transfer learning tutorial".
https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

This research used resources of the Oak Ridge Leadership Computing Facility at the Oak Ridge National Laboratory, which is supported by the Advanced Scientific Computing Research programs in the Office of Science of the U.S. Department of Energy under Contract No. DE-AC05-00OR22725.

This material is based upon work supported by the U.S. Department of Energy, Office of Science, National Quantum Information Science Research Centers, Quantum Science Center.



OAK RIDGE

National Laboratory

LEADERSHIP
COMPUTING
FACILITY



U.S. DEPARTMENT
of **ENERGY**

ORNL IS MANAGED BY UT-BATTELLE LLC
FOR THE US DEPARTMENT OF ENERGY